UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS – CMPF
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIA
DISCIPLINA: PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

QUESTÕES RESPONDIDAS UNIDADE II

FERNANDO UMBILINO ALVES

PAU DOS FERROS – RN
2025

QUESTÕES RESPONDIDAS

- Questão 20;
- Questão 21;
- Questão 22;
- Questão 23;
- Questão 24;
- Questão 25;
- Questão 26;
- Questão 28;
- Questão 30;
- Questão 32;

Programação de memória compartilhada com OpenMP

Questão 20: Baixe o arquivo *omp_trap_1.c* do site do livro e exclua a diretiva critical. Compile e execute o programa com cada vez mais threads e valores cada vez maiores de *n*.

```
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ gcc -g -Wall -fopenmp -o omp_trap_1 omp_trap_1.c
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ ./omp_trap_1 1
Enter a, b, and n
10 52 256
With n = 256 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65361884155273e+04
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ ./omp_trap_1 1
Enter a, b, and n
10 52 512
With n = 512 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65360471038818e+04
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ ./omp_trap_1 1
Enter a, b, and n
10 52 1024
With n = 1024 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65360117759705e+04
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ ./omp_trap_1 1
Enter a, b, and n
10 52 2048
With n = 2048 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65360029439926e+04
fernando@ubuntu:~/Documentos/PCD/ipp-source/ipp-source-use/ch5/q20$ ./omp_trap_1 4
Enter a, b, and n
10 52 256
With n = 256 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 3.94775163116455e+04
```

(a) Quantas threads e quantos trapézios são necessários antes que o resultado esteja incorreto?

Com a exclusão da diretiva critical, a partir de 2 threads já ocorre condições de corrida, porém, baseado nos parâmetros e testes acima, com 4 threads e 256 trapézios o valor foi incorreto.

(b) Como o aumento do número de trapézios influencia nas chances do resultado ser incorreto?

À medida que aumenta o número de trapézio, as threads vão fazer mais interações e consequentemente realizar acesso na variável *global_result* mais vezes, aumentando as chances do resultado ser incorreto.

(c) Como o aumento do número de threads influencia nas chances do resultado ser incorreto?

Com o aumento do número de threads, maiores as chances de ter condição de corrida e consequentemente realizar acesso na variável *global_result* mais vezes, aumentando as chances do resultado ser incorreto.

Questão 21: Baixe o arquivo *omp_trap1.c* do site do livro. Modifique o código para que

- ele use o primeiro bloco de código da página 222 do livro e
- o tempo usado pelo bloco paralelo seja cronometrado usando a função OpenMP *omp_get_wtime()*. A sintaxe é *double omp_get_wtime(void)*

Ele retorna o número de segundos que se passaram desde algum tempo no passado. Para obter detalhes sobre cronometragem, consulte a Seção 2.6.4. Lembre-se também de que o OpenMP possui uma diretiva de barreira:

 # pragma omp barrier

Agora encontre um sistema com pelo menos dois núcleos e cronometre o programa com

- uma *thread* e um grande valor de *n*, e
- duas *threads* e o mesmo valor de *n*.

```C/C++
//antes função Trap
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
}  /* Trap */
```

```C/C++
//depois função Trap
double Trap(double a, double b, int n) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
```

```
      x = local_a + i*h;
      my_result += f(x);
   }
   my_result = my_result*h;

   return my_result;
}  /* Trap */
```

C/C++

```
//antes na Main
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

C/C++

```
//depois na Main
#  pragma omp parallel num_threads(thread_count) default (none)
shared(global_result, a, b, n, time_result) private(start_time, finish_time)
   {
#  pragma omp barrier
   start_time = omp_get_wtime();
# pragma omp critical (result)
   global_result += Trap(a, b, n);
   finish_time = omp_get_wtime();
   int thead_number = omp_get_thread_num();
   printf("Thread %d Time: %f\n", thead_number, (finish_time - start_time));

   if (time_result < (finish_time - start_time)) {
#     pragma omp critical (time)
      time_result = (finish_time - start_time);
   }
   }

   printf ("Tempo: %f\n", time_result);
   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n", a, b, global_result);
   return 0;
}  /* main */
```

(a) O que acontece?

Os prints abaixo são da mediana de 5 execuções do tempo de cada Thread.

Tempos com 1 thread: 3.599637; 3.600174; 3.611418; 3.618698; 3.641462

Tempos com 2 threads: 3.600549; 3.615978; 3.622204; 3.628246; 3.628884

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia/q21$ ./omp_trap1 1
Enter a, b, and n
10 52 1000000000
Thread 0 Time: 3.611418
Tempo: 3.611418
With n = 1000000000 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65359999999955e+04
```

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia/q21$ ./omp_trap1 2
Enter a, b, and n
10 52 1000000000
Thread 0 Time: 1.829800
Thread 1 Time: 3.622204
Tempo: 3.622204
With n = 1000000000 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65359999999985e+04
```

Devido a *omp_critical*, a função Trap é acessada sequencialmente, por causa disso o tempo de execução com 2 trhreads foi maior do que o tempo com 1 thread.

(b) Baixe o arquivo *omp_trap2b.c* do site do livro. Como seu desempenho se compara?

Os prints abaixo são da mediana de 5 execuções do tempo de cada Thread.

Tempos com 1 thread: 3.559610; 3.563623; 3.565847; 3.569103; 3.573454

Tempos com 2 threads: 1.800115; 1.807263; 1.813070; 1.823964; 1.845232

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap2b 1
Enter a, b, and n
10 52 1000000000
Thread 0 Time: 3.565847
Time result: 3.565847
With n = 1000000000 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65359999999955e+04
```

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap2b 2
Enter a, b, and n
10 52 1000000000
Thread 0 Time: 1.812408
Thread 1 Time: 1.813070
Time result: 1.813070
With n = 1000000000 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65359999999985e+04
```

O desempenho melhorou significativamente com 2 threads, nesta versão do código, a chamada sequencial feita pela *omp_critical* foi eliminada e substituída por uma redução global, utilizando a cláusula *reduction,* dessa forma a threads executam paralelamente.

Questão 22. Suponha que no incrível computador Bleeblon, variáveis com tipo float possam armazenar três dígitos decimais. Suponha também que os registradores de ponto flutuante do Bleeblon possam armazenar quatro dígitos decimais e que, após qualquer operação de ponto flutuante, o resultado seja arredondado para três dígitos decimais antes de ser armazenado. Agora suponha que um programa C declare um *array a* da seguinte forma:

float a[] = {4.0, 3.0, 3.0, 1000.0};

(a) Qual é a saída do seguinte bloco de código se ele for executado no Bleeblon? Justifique sua resposta.

```
int i ;
float sum = 0.0;
for (i = 0; i < 4; i++)
    sum += a[i];
printf ("sum = %4.1f\n", sum );
```

1 interação: sum = 0.0; i = 0; a[0] = 4.0

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	0.000×10^0	4.000×10^0	
2	Comparar expoentes	0.000×10^0	4.000×10^0	
3	Deslocar um operando	0.000×10^0	4.000×10^0	
4	Somar	0.000×10^0	4.000×10^0	4.000×10^0
5	Normalizar resultado	0.000×10^0	4.000×10^0	4.000×10^0
6	Arredondar resultado	0.000×10^0	4.000×10^0	4.00×10^0
7	Armazenar resultado	0.000×10^0	4.000×10^0	4.00×10^0

2 interação: sum = 4.0; i = 1; a[1] = 3.0

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	4.000×10^0	3.000×10^0	
2	Comparar expoentes	4.000×10^0	3.000×10^0	
3	Deslocar um operando	4.000×10^0	3.000×10^0	
4	Somar	4.000×10^0	3.000×10^0	7.000×10^0
5	Normalizar resultado	4.000×10^0	3.000×10^0	7.000×10^0
6	Arredondar resultado	4.000×10^0	3.000×10^0	7.00×10^0
7	Armazenar resultado	4.000×10^0	3.000×10^0	7.00×10^0

3 interação: sum = 7.0; i = 2; a[2] = 3.0

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	7.000×10^0	3.000×10^0	
2	Comparar expoentes	7.000×10^0	3.000×10^0	
3	Deslocar um operando	7.000×10^0	3.000×10^0	
4	Somar	7.000×10^0	3.000×10^0	10.00×10^0
5	Normalizar resultado	7.000×10^0	3.000×10^0	1.000×10^1
6	Arredondar resultado	7.000×10^0	3.000×10^0	1.00×10^1
7	Armazenar resultado	7.000×10^0	3.000×10^0	1.00×10^1

4 interação: sum = 10.0; i = 3; a[3] = 1000.0

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	1.000×10^1	1.000×10^3	
2	Comparar expoentes	1.000×10^1	1.000×10^3	
3	Deslocar um operando	0.010×10^3	1.000×10^3	
4	Somar	0.010×10^3	1.000×10^3	1.010×10^3
5	Normalizar resultado	0.010×10^3	1.000×10^3	1.010×10^3
6	Arredondar resultado	0.010×10^3	1.000×10^3	1.01×10^3
7	Armazenar resultado	0.010×10^3	1.000×10^3	1.01×10^3

Resultado final: sum = 1010.0

(b) Agora considere o seguinte código:

```
int i;
float sum = 0.0;
#pragma omp parallel for num threads (2) reduction (+:sum)
for (i = 0; i < 4; i++)
    sum += a[i];
printf("sum = %4.1f\n", sum );
```

Suponha que o sistema operacional atribua as iterações *i = 0, 1* à *thread 0* e *i = 2, 3* à *thread 1*. Qual é a saída deste código no Bleeblon? Justifique sua resposta.

Na Thread 0:

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	4.000×10^0	3.000×10^0	
2	Comparar expoentes	4.000×10^0	3.000×10^0	
3	Deslocar um operando	4.000×10^0	3.000×10^0	
4	Somar	4.000×10^0	3.000×10^0	7.000×10^0
5	Normalizar resultado	4.000×10^0	3.000×10^0	7.000×10^0
6	Arredondar resultado	4.000×10^0	3.000×10^0	7.00×10^0
7	Armazenar resultado	4.000×10^0	3.000×10^0	7.00×10^0

Na Thread 1:

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	3.000×10^0	1.000×10^3	
2	Comparar expoentes	3.000×10^0	1.000×10^3	
3	Deslocar um operando	0.003×10^3	1.000×10^3	
4	Somar	0.003×10^3	1.000×10^3	1.003×10^3
5	Normalizar resultado	0.003×10^3	1.000×10^3	1.003×10^3
6	Arredondar resultado	0.003×10^3	1.000×10^3	1.00×10^3
7	Armazenar resultado	0.003×10^3	1.000×10^3	1.00×10^3

Na Reduction:

Tempo	Operação	Operando 1	Operando 2	Resultado
1	Buscar operandos	7.000×10^0	1.000×10^3	
2	Comparar expoentes	7.000×10^0	1.000×10^3	
3	Deslocar um operando	0.007×10^3	1.000×10^3	
4	Somar	0.007×10^3	1.000×10^3	1.007×10^3

5	Normalizar resultado	0.007×10^3	1.000×10^3	1.007×10^3
6	Arredondar resultado	0.007×10^3	1.000×10^3	1.00×10^3
7	Armazenar resultado	0.007×10^3	1.000×10^3	1.00×10^3

Resultado final: sum = 1000.0
Antes de armazenar o resultado, ele trunca para 3 dígitos, tanto na thread 1 como na reductions, fazendo o valor que foi armazenado ser incorreto, com isso o resultado final fica errado.

Questão 23. Escreva um programa OpenMP que determine o escalonamento padrão de laços *for* paralelos. Sua entrada deve ser o número de iterações e quantidade de *threads* e sua saída deve ser quais iterações de um laço *for* paralelizado são executadas por qual *thread*. Por exemplo, se houver duas *threads* e quatro iterações, a saída poderá ser:

Thread 0: Iterações 0 -- 1
Thread 1: Iterações 2 -- 3

```
C/C++
#   pragma omp parallel num_threads(number_of_threads) \
    default(none) private(i, my_rank) shared(number_of_interactions)
    {
#   pragma omp for
    for(i = 0; i < number_of_interactions; i++){
        my_rank = omp_get_thread_num();
        printf("Thread: %d, interaction: %d\n", my_rank, i);
    }
    }
```

(a) De acordo com a execução do seu programa, qual é o escalonamento padrão de laços *for* paralelos de um programa OpenMP? Porque?
Executado com 4 threads e 16 interações

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./q23 4 16
Thread: 1, interaction: 4
Thread: 1, interaction: 5
Thread: 1, interaction: 6
Thread: 1, interaction: 7
Thread: 3, interaction: 12
Thread: 3, interaction: 13
Thread: 3, interaction: 14
Thread: 3, interaction: 15
Thread: 2, interaction: 8
Thread: 2, interaction: 9
Thread: 2, interaction: 10
Thread: 2, interaction: 11
Thread: 0, interaction: 0
Thread: 0, interaction: 1
Thread: 0, interaction: 2
Thread: 0, interaction: 3
```

De acordo com o teste, o tipo é o *static*, devido ao trabalho ter sido dividido igualmente entre as threads, e o tamanho do bloco foi o resultado entre interações/threads (16/4) que foi 4.

Questão 24: Considere o seguinte laço:

```
a[0] = 0;
for ( i = 1; i < n ; i++)
a[i] = a[i-1] + i;
```

Há claramente uma dependência no laço já que o valor de *a[i]* não pode ser calculado sem o valor de *a[i-1]*. Sugira uma maneira de eliminar essa dependência e paralelizar o laço.

Analisando o laço e valor inicial atribuído a a[0] = 0, foi possível compreender que o laço realiza o somatório de uma sequência de números inteiros. O laço calcula:

- a[0] = 0
- a[1] = a[0] + 1 = 0 + 1 = 1
- …
- a[i] = 0 + 1 + i + … + i

Essa soma possui uma fórmula de forma fechada, que pode ser calculada substituindo "a[i] = a[i-1] + i;" por "a [i] = (i*(i + 1))/2;", assim é removido a dependência, dessa maneira não é preciso recuperar o valor anterior, já que a fórmula faz o cálculo apenas com o valor do índice atual.

Questão 25: Modifique o programa da regra do trapézio que usa uma diretiva *parallel for (omp_trap_3.c)* para que o *parallel for* seja modificado por uma cláusula *schedule(runtime)*. Execute o programa com várias atribuições à variável de ambiente *OMP_SCHEDULE* e determine quais iterações são atribuídas a qual *thread*. Isso pode ser feito alocando um array *iteracoes* de *n int*'s e, na função *Trap*, atribuindo *omp_get_thread_num()* a *iteracoes[i]* na *i*-ésima iteração do laço *for*. Qual é o escalonamento padrão de iterações em seu sistema? Como o escalonamento *guided* é determinado?

```C/C++
//antes
double Trap(double a, double b, int n, int thread_count) {
   double  h, approx;
   int  i;

   h = (b-a)/n;
   approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx)
   for (i = 1; i <= n-1; i++)
     approx += f(a + i*h);
   approx = h*approx;

   return approx;
}  /* Trap */
```

```C/C++
//depois
double Trap(double a, double b, int n, int thread_count) {
   double  h, approx;
   int  i;
   int *interation = (int *)malloc(n * sizeof(int));

   h = (b-a)/n;
   approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx) schedule(runtime)
   for (i = 1; i <= n-1; i++) {
     approx += f(a + i*h);
     interation[i] = omp_get_thread_num();
     printf("Thread %d: Interação %d\n", interation[i], i);
   }
     approx = h*approx;

   free(interation);

   return approx;
}  /* Trap */
```

Executei o programa sem definir à variável de ambiente *OMP_SCHEDULE*, onde o resultado foi:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap3 4
Enter a, b, and n
10 52 16
Thread 2: Interação 2
Thread 2: Interação 5
Thread 2: Interação 6
Thread 2: Interação 7
Thread 2: Interação 8
Thread 2: Interação 9
Thread 2: Interação 10
Thread 2: Interação 11
Thread 2: Interação 12
Thread 2: Interação 13
Thread 2: Interação 14
Thread 2: Interação 15
Thread 1: Interação 1
Thread 3: Interação 3
Thread 0: Interação 4
With n = 16 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65842343750000e+04
```

De acordo com o teste, o tipo padrão de escalonamento é o *"dynamic, 1"*, atribuiu as iterações às threads em blocos fixos de 1, conforme as threads ficavam ociosas. É notável que a Thread 2 foi a que mais rapidamente finalizou suas iterações após a 4ª, retornando para adquirir um próximo bloco de trabalho mais vezes.

Executei o programa com várias atribuições à variável de ambiente *OMP_SCHEDULE*, onde os resultados foram:

OMP_SCHEDULE = "static, 2"

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ export OMP_SCHEDULE="static, 2"
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap3 4
Enter a, b, and n
10 52 16
Thread 3: Interação 7
Thread 3: Interação 8
Thread 3: Interação 15
Thread 1: Interação 3
Thread 1: Interação 4
Thread 1: Interação 11
Thread 1: Interação 12
Thread 2: Interação 5
Thread 2: Interação 6
Thread 2: Interação 13
Thread 2: Interação 14
Thread 0: Interação 1
Thread 0: Interação 2
Thread 0: Interação 9
Thread 0: Interação 10
With n = 16 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65842343750000e+04
```

Neste exemplo, com tamanho de bloco de dois, o escalonamento *static* dividiu as iterações igualmente entre as threads em blocos fixos de dois.

OMP_SCHEDULE = "dynamic, 2"

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ export OMP_SCHEDULE="dynamic, 2"
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap3 4
Enter a, b, and n
10 52 16
Thread 1: Interação 1
Thread 1: Interação 2
Thread 1: Interação 9
Thread 1: Interação 10
Thread 1: Interação 11
Thread 1: Interação 12
Thread 1: Interação 13
Thread 1: Interação 14
Thread 1: Interação 15
Thread 2: Interação 5
Thread 2: Interação 6
Thread 0: Interação 7
Thread 3: Interação 3
Thread 3: Interação 4
Thread 0: Interação 8
With n = 16 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65842343750000e+04
```

Neste exemplo, com um tamanho de bloco de dois, o escalonamento *dynamic* atribuiu as iterações às threads em blocos fixos de dois, conforme as threads ficavam ociosas.

OMP_SCHEDULE = "auto"

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ export OMP_SCHEDULE="auto, 2"
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap3 4
Enter a, b, and n
10 52 16
Thread 2: Interação 9
Thread 2: Interação 10
Thread 2: Interação 11
Thread 2: Interação 12
Thread 3: Interação 13
Thread 3: Interação 14
Thread 3: Interação 15
Thread 0: Interação 1
Thread 0: Interação 2
Thread 0: Interação 3
Thread 0: Interação 4
Thread 1: Interação 5
Thread 1: Interação 6
Thread 1: Interação 7
Thread 1: Interação 8
With n = 16 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65842343750000e+04
```

Neste exemplo, o escalonamento *auto* foi utilizado, permitindo que o sistema OpenMP otimizasse a distribuição das iterações, escolhendo a estratégia e os tamanhos de bloco mais adequados em tempo de execução. Nesse caso foi *static* de *tamanho* 4.

OMP_SCHEDULE = "guided, 2"

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ export OMP_SCHEDULE="guided, 2"
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./omp_trap3 4
Enter a, b, and n
10 52 16
Thread 3: Interação 8
Thread 3: Interação 9
Thread 3: Interação 12
Thread 3: Interação 13
Thread 3: Interação 14
Thread 3: Interação 15
Thread 0: Interação 10
Thread 0: Interação 11
Thread 2: Interação 1
Thread 2: Interação 2
Thread 2: Interação 3
Thread 2: Interação 4
Thread 1: Interação 5
Thread 1: Interação 6
Thread 1: Interação 7
With n = 16 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.65842343750000e+04
```

Neste exemplo, com um tamanho de bloco mínimo de dois, o escalonamento *guided* distribuiu as iterações de forma dinâmica, começando com blocos maiores e diminuindo seu tamanho à medida que o trabalho restante diminuía.
O *guided* busca equilibrar a carga de trabalho entre as threads, essa é sua característica, no começo da execução, as threads recebem blocos relativamente grandes, isso reduz a frequência com que as threads precisam se comunicar com o runtime para pegar mais trabalho, à medida que o laço progride e a quantidade total de trabalho restante diminui, o tamanho atribuídos às threads também diminui.

Questão 26: Lembre-se de que todos os blocos estruturados modificados por uma diretiva critical formam uma única secção crítica. O que acontece se tivermos um número de diretivas *atomic* nas quais diferentes variáveis estão sendo modificadas? Todas elas são tratadas como uma única seção crítica?
Podemos escrever um pequeno programa que tente determinar isso. A ideia é fazer com que todas as threads executem simultaneamente algo como o código a seguir:

```
        int i;
        double minha_soma = 0.0;
        for (i = 0; i < n; i++)
        #pragma omp atomic
        minha_soma += sin(i);
```

Podemos fazer isso modificando o código com uma diretiva parallel:

```
    # pragma omp parallel num threads(thread_count)
    {
        int i;
        double minha_soma = 0.0;
        for (i = 0; i < n; i ++)
    # pragma omp atomic
        minha_soma += sin( i);
    }
```

Observe que já que minha_soma e i são declaradas no bloco paralelo, cada thread possui sua própria cópia privada. Agora, se medirmos o tempo desse código para um n grande com thread_count = 1 e também quando thread_count > 1, contanto que thread_count seja menor que o número de núcleos disponíveis, o tempo de execução para a execução de thread única deveria ser aproximadamente o mesmo que o tempo para a execução com múltiplas threads se as diferentes execuções de minha_soma += sin(i) são tratadas como diferentes seções críticas. Por outro lado, se as diferentes execuções de minha_soma += sin(i) são todas tratadas como uma única seção crítica, a execução com múltiplas threads deve ser muito mais lenta que a execução de thread única. Escreva um programa OpenMP que implemente este teste. Sua implementação do OpenMP permite a execução simultânea de atualizações para diferentes variáveis quando as atualizações são protegidas por diretivas atomic?

O *atomic* garante uma única operação de memória em uma variável específica. Se diferentes variáveis estão sendo modificadas por diretivas *atomic*, essas operações não são tratadas como uma única seção crítica e podem ser executadas paralelamente.

```
C/C++
#  pragma omp parallel num_threads(thread_count) default (none) shared
(time_result, n)
   {
    double start_time, finish_time;
    int i;
    double minha_soma = 0.0;

    start_time = omp_get_wtime();
    for (i = 1; i <= n; i ++) {
#       pragma omp atomic
        minha_soma += sin(i);
    }
    finish_time = omp_get_wtime();

    int thead_number = omp_get_thread_num();
```

```
 printf("Thread %d - Soma: %.2f\n", thead_number, minha_soma);
 printf("Thread %d Time: %f\n", thead_number, (finish_time-start_time));

if (time_result < (finish_time - start_time)) {
      time_result = (finish_time - start_time);
}
}

printf ("Tempo: %f\n", time_result);
```

Executei o programa com n = 100.000.000 para 1 thread e 4 threads, onde os resultados foram:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./q26 1
Enter number interations
100000000
Thread 0 - Soma: 1.71
Thread 0 Time: 1.721712
Tempo: 1.721712
```

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia$ ./q26 4
Enter number interations
100000000
Thread 1 - Soma: 1.71
Thread 1 Time: 2.693281
Thread 0 - Soma: 1.71
Thread 0 Time: 2.698836
Thread 2 - Soma: 1.71
Thread 2 Time: 2.760644
Thread 3 - Soma: 1.71
Thread 3 Time: 2.762939
Tempo: 2.762939
```

De acordo com as imagens acima, o tempo de execução com 1 thread para 4 threads foi bastante pequeno, podemos concluir que implementação do OpenMP permite a execução simultânea de atualizações para diferentes variáveis quando as atualizações são protegidas por diretivas *atomic*.

Questão 28: Lembre-se do exemplo de multiplicação de matrizes e vetores com a entrada 8000 × 8000. Assuma que uma linha de cache contém 64 bytes ou 8 doubles.

(a) Suponha que a thread 0 e a thread 2 sejam atribuídas a processadores diferentes. É possível que ocorra um falso compartilhamento entre as threads 0 e 2 para alguma parte do vetor y? Por que?

Sim, é possível ocorrer falso compartilhamento entre as threads 0 e a thread 2. Assumindo que a linha de cache possui 8 doubles, se o bloco do escalonamento for por exemplo *"static,2"*, as threads são atribuídas a trabalhar em partes do vetor *y* que são muito próximas na memória, dentro da mesma linha de cache. Isso leva a um falso compartilhamento significativo, pois as operações de escrita de uma thread invalidarão repetidamente a linha de cache para as outras threads, degradando o desempenho.

(b) E se a thread 0 e a thread 3 forem atribuídas a processadores diferentes? É possível que ocorra um falso compartilhamento entre elas para alguma parte de y?

Sim, analogamente à alternativa (a), também existe a possibilidade de ocorrer o falso compartilhamento entre a thread 0 e a thread 3, seguindo o mesmo exemplo do escalonamento do tipo *"static,2"*.

Questão 30. Utilizando OpenMP, implemente o programa paralelo do histograma discutido no Capítulo 2.

Foi usado como base para o programa, o arquivo histograma.c do capítulo 2

```
C/C++
int main(int argc, char* argv[]) {
   int bin_count = 20;
   int i, bin, thread_count, thread_number;
   float min_meas, max_meas;
   float bin_maxes[20];
   int bin_counts[20];
   int data_count;
   float* data;

   /* Check and get command line args */
   if (argc != 5) Usage(argv[0]);
   Get_args(argv, &min_meas, &max_meas, &data_count, &thread_count);

   /* Allocate arrays needed */
   data = malloc(data_count*sizeof(float));

   /* Generate the data */
   Gen_data(min_meas, max_meas, data, data_count);

   /* Create bins for storing counts */
   Gen_bins(min_meas, max_meas, bin_maxes, bin_counts, bin_count);

   /* Count number of values in each bin */
#  pragma omp parallel num_threads(thread_count) \
   reduction(+: bin_counts) default(none) \
   shared(data, bin_maxes, data_count, bin_count, min_meas) private(i, bin,
thread_number)
      {

#     pragma omp for
         for (i = 0; i < data_count; i++) {
            bin = Which_bin(data[i], bin_maxes, bin_count, min_meas);
            thread_number = omp_get_thread_num();
            //printf("Thread number: %d -> Value: %.2f \n", thread_number,
data[i]);
            bin_counts[bin]++;
         }
   }
```

```
#  ifdef DEBUG
   printf("bin_counts = ");
   for (i = 0; i < bin_count; i++)
      printf("%d ", bin_counts[i]);
   printf("\n");
#  endif

   /* Print the histogram */
   Print_histo(bin_maxes, bin_counts, bin_count, min_meas);

   free(data);
   return 0;

}  /* main */
```

Questão 32. *Count sort* é um algoritmo de ordenação serial simples que pode ser implementado da seguinte forma:

```
void Count_sort(int a[], int n) {
        int i, j, count;
        int* temp = malloc(n*sizeof(int));
        for (i = 0; i < n; i++) {
                count = 0;
                for (j = 0; j < n; j++)
                        if (a[j] < a [i])
                                count++;
                        else if (a[j] == a[i] && j < i)
                                count++;
                temp[count] = a[i];
        }
        memcpy(a, temp, n*sizeof(int));
        free(temp);
        }
```

A ideia básica é que para cada elemento *a[i]* na lista a, contemos o número de elementos da lista que são menores que *a[i]*. Em seguida, inserimos *a[i]* em uma lista temporária usando o índice determinado pela contagem. Há um pequeno problema com esta abordagem quando a lista contém elementos iguais, uma vez que eles podem ser atribuídos ao mesmo slot na lista temporária. O código lida com isso incrementando a contagem de elementos iguais com base nos índices. Se *a[i]* == *a[j]* e *j* < *i*, então contamos *a[j]* como sendo "menor que" *a[i]*.
Após a conclusão do algoritmo, sobrescrevemos o array original pelo array temporário usando a função da biblioteca de strings *memcpy*.

(a) Se tentarmos paralelizar o laço *for i* (o laço externo), quais variáveis devem ser privadas e quais devem ser compartilhadas?
Private (i, j, count)

Shared (temp, n, a)

(b) Se paralelizarmos o laço *for i* usando o escopo especificado na parte anterior, haverá alguma dependência de dados no laço? Explique sua resposta.
Não haverá dependência de dados, pois para cada iteração do laço externo *i* realiza um trabalho isolado para cada thread, cada interação lê o array *a[ ],* calcula uma posição *count* e escreve o elemento *a[i]* na posição *temp[count]*. Como a posição *count* é única para cada *a[i]* que está sendo colocado em *temp*, as operações de escrita são independentes e não há dependência de dados entre as iterações do laço externo *i*.

(c) Podemos paralelizar a chamada para *memcpy*? Podemos modificar o código para que esta parte da função seja paralelizável?
Sim, é possível paralelizar. A diretiva *#pragma omp barrier* é crucial, pois garante que todas as threads tenham concluído a fase de ordenação antes que qualquer thread comece a ler de *temp* para copiar para *a*, as variáveis *start* e *interval* são calculadas para cada thread, implementando uma estratégia de distribuição de blocos, a chamada *memcpy(&a[start], &temp[start], interval*sizeof(int))* é então executada em paralelo por cada thread, onde cada thread opera em um segmento de memória diferente.

```C/C++
#  pragma omp barrier
      int rest = n % thread_count;
      int thread_number = omp_get_thread_num();
      int interval, start;
      if(rest == 0){
         interval = n / thread_count;
         start = thread_number * interval;
      }else{
         if (thread_number < rest){
            interval = (n / thread_count) + 1;
            start = (thread_number * (n/thread_count)) + thread_number;
         }else{
            interval = (n / thread_count);
            start = (thread_number * interval) + rest;
         }
      }

      memcpy(&a[start], &temp[start], interval*sizeof(int));
   }
```

(d) Escreva um programa em C que inclua uma implementação paralela do *Count sort*.

```C/C++
void Count_sort(int a[], int n, int thread_count) {
```

```
int i, j, count;
int *temp = malloc(n*sizeof(int));

#  pragma omp parallel num_threads(thread_count) \
   default (none) shared(temp, a, n, thread_count) private(count, i, j)
   {
#     pragma omp for
      for (i = 0; i < n; i++) {
         count = 0;
         for (j = 0; j < n; j++){
            if (a[j] < a [i])
               count++;
            else if (a[j] == a[i] && j < i)
               count++;
         }
      temp[count] = a[i];
      }

#  pragma omp barrier
      int rest = n % thread_count;
      int thread_number = omp_get_thread_num();
      int interval, start;
      if(rest == 0){
         interval = n / thread_count;
         start = thread_number * interval;
      }else{
         if (thread_number < rest){
            interval = (n / thread_count) + 1;
            start = (thread_number * (n/thread_count)) + thread_number;
         }else{
            interval = (n / thread_count);
            start = (thread_number * interval) + rest;
         }
      }

      memcpy(&a[start], &temp[start], interval*sizeof(int));
   }


free(temp);
} /*Count_sort*/
```

(e) Como o desempenho da sua paralelização do *Count sort* se compara à classificação serial? Como ela se compara à função serial *qsort*?

Em tempo de execução, o *Count Sort paralelo* demonstrou ser mais rápida que a sua versão *serial* (22.47s vs. 53.52s, respectivamente, para 100.000 elementos). Contudo,

ambos os métodos de *Count Sort* são significativamente mais lentos que a função de biblioteca *qsort* (0.02s), devido a complexidade ser menor do que as versẽs de *Count Sort*.

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia/q32$ ./omp_parel_32 4
Enter the amount of data
100000
Total time parallel: 22.467765
```

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch5-copia/q32$ ./omp_32 4
Enter the amount of data
100000
Total time sequencial: 53.520757
Total time qsort: 0.019520
```